

**R. Uday Kumar Reddy (192424190)****C1 Lab Experiments**

1. Given an array of strings words, return the first palindromic string in the array. If there is no such string, return an empty string "". A string is palindromic if it reads the same forward and backward.

Example 1:

Input: words = ["abc","car","ada","racecar","cool"]

Output: "ada"

Explanation: The first string that is palindromic is "ada".

Note that "racecar" is also palindromic, but it is not the first.

Example 2:

Input: words = ["notapalindrome","racecar"]

Output: "racecar"

Explanation: The first and only string that is palindromic is "racecar".

1.

```
words = ["abc", "car", "ada", "racecar", "cool"]
for word in words:
    if word == word[::-1]:
        print("First Palindrome:", word)
        break
    else:
        print("")
```

... First Palindrome: ada

2. You are given two integer arrays nums1 and nums2 of sizes n and m, respectively. Calculate the following values: answer1 : the number of indices i such that nums1[i] exists in nums2. answer2 : the number of indices i such that nums2[i] exists in nums1 Return [answer1,answer2].

Example 1:

Input: nums1 = [2,3,2], nums2 = [1,2]

Output: [2,1]

Explanation:

2.

```
nums1 = [2, 3, 2]
nums2 = [1, 2]
answer1 = 0
answer2 = 0
for x in nums1:
    if x in nums2:
        answer1 += 1
for x in nums2:
    if x in nums1:
        answer2 += 1
print([answer1, answer2])
```

... [2, 1]

3. You are given a 0-indexed integer array `nums`. The distinct count of a subarray of `nums` is defined as: Let `nums[i..j]` be a subarray of `nums` consisting of all the indices from `i` to `j` such that  $0 \leq i \leq j < \text{nums.length}$ . Then the number of distinct values in `nums[i..j]` is called the distinct count of `nums[i..j]`. Return the sum of the squares of distinct counts of all subarrays of `nums`. A subarray is a contiguous non-empty sequence of elements within an array

3.

---

```
nums = [1, 2, 1]
total = 0
for i in range(len(nums)):
    s = set()
    for j in range(i, len(nums)):
        s.add(nums[j])
        total += len(s) ** 2
print("Answer:", total)
```

---

Answer: 15

---

4. Given a 0-indexed integer array `nums` of length `n` and an integer `k`, return *the number of pairs*  $(i, j)$  *where*  $0 \leq i < j < n$ , *such that* `nums[i] == nums[j]` *and*  $(i * j)$  *is divisible by* `k`.

4.

---

```
nums = [3, 1, 2, 2, 2, 1, 3]
k = 2
count = 0
for i in range(len(nums)):
    for j in range(i + 1, len(nums)):
        if nums[i] == nums[j] and (i * j) % k == 0:
            count += 1
print("Pairs:", count)
```

---

• Pairs: 4

---

5. Write a program FOR THE BELOW TEST CASES with least time complexity

5.

---

```
arr = [1, 2, 3, 4, 5]
maximum = arr[0]
for num in arr:
    if num > maximum:
        maximum = num
print("Maximum:", maximum)
```

---

Maximum: 5

---

6. You have an algorithm that process a list of numbers. It firsts sorts the list using an efficient sorting algorithm and then finds the maximum element in sorted list. Write the code for the same.

6.

---

```
arr = [5, 2, 8, 1, 9]
if len(arr) == 0:
    print("List is empty")
else:
    arr.sort()
    print("Sorted List:", arr)
    print("Maximum Element:", arr[-1])
```

---

Sorted List: [1, 2, 5, 8, 9]  
Maximum Element: 9

---

7. Write a program that takes an input list of n numbers and creates a new list containing only the unique elements from the original list. What is the space complexity of the algorithm?

7.

---

```
▶ nums = [3, 7, 3, 5, 2, 5, 9, 2]
  unique = []
  for x in nums:
      if x not in unique:
          unique.append(x)
  print("Unique Elements:", unique)
```

---

```
.. Unique Elements: [3, 7, 5, 2, 9]
```

---

8. Sort an array of integers using the bubble sort technique. Analyze its time complexity using Big-O notation. Write the code

8.

---

```
▶ arr = [64, 34, 25, 12, 22, 11, 90]
  n = len(arr)
  for i in range(n):
      for j in range(0, n - i - 1):
          if arr[j] > arr[j + 1]:
              arr[j], arr[j + 1] = arr[j + 1], arr[j]
  print("Sorted Array:", arr)
```

---

```
.. Sorted Array: [11, 12, 22, 25, 34, 64, 90]
```

---

9. Checks if a given number  $x$  exists in a sorted array `arr` using binary search. Analyze its time complexity using Big-O notation.

9.

```
➤ arr = [-9, 3, 4, 6, 8, 9, 10, 30]
  key = 10
  low = 0
  high = len(arr) - 1
  found = False
  while low <= high:
      mid = (low + high) // 2

      if arr[mid] == key:
          print("Element found at position", mid + 1)
          found = True
          break

      elif arr[mid] < key:
          low = mid + 1

      else:
          high = mid - 1

  if not found:
      print("Element not found")

.. Element found at position 7
```

10. Given an array of integers `nums`, sort the array in ascending order and return it. You must solve the problem without using any built-in functions in  $O(n \log(n))$  time complexity and with the smallest space complexity possible.

10.

```
➤ def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left = arr[:mid]
        right = arr[mid:]
        merge_sort(left)
        merge_sort(right)
        i = j = k = 0
        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1
        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1
        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1
    arr = [5, 2, 8, 1, 3]
    merge_sort(arr)
    print("Sorted Array:", arr)

.. Sorted Array: [1, 2, 3, 5, 8]
```

11. Given an  $m \times n$  grid and a ball at a starting cell, find the number of ways to move the ball out of the grid boundary in exactly  $N$  steps.

11.

---

```
def findPaths(m, n, N, i, j):
    dp = [[0] * n for _ in range(m)]
    dp[i][j] = 1
    count = 0
    for step in range(N):
        temp = [[0] * n for _ in range(m)]
        for r in range(m):
            for c in range(n):
                if dp[r][c] > 0:
                    for dr, dc in [(-1,0), (1,0), (0,-1), (0,1)]:
                        nr = r + dr
                        nc = c + dc

                        if nr < 0 or nr >= m or nc < 0 or nc >= n:
                            count += dp[r][c]
                        else:
                            temp[nr][nc] += dp[r][c]

        dp = temp
    return count
print(findPaths(2, 2, 2, 0, 0))
print(findPaths(1, 3, 3, 0, 1))
```

---

... 6  
12

12. You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. All houses at this place are arranged in a circle. That means the first house is the neighbor of the last one. Meanwhile, adjacent houses have security systems connected, and it will automatically contact the police if two adjacent houses were broken into on the same night.

12.

---

```
def rob_linear(nums):
    prev = 0
    curr = 0
    for money in nums:
        prev, curr = curr, max(curr, prev + money)
    return curr
def rob(nums):
    if len(nums) == 1:
        return nums[0]
    return max(
        rob_linear(nums[:-1]),
        rob_linear(nums[1:])
    )
nums = [2, 3, 2]
print(rob(nums))

nums = [1, 2, 3, 1]
print(rob(nums))
```

---

... 3  
4

13. You are climbing a staircase. It takes  $n$  steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

13.

---

```
def climbStairs(n):  
    if n <= 2:  
        return n  
    first = 1  
    second = 2  
    for i in range(3, n + 1):  
        third = first + second  
        first = second  
        second = third  
    return second  
print(climbStairs(4))  
print(climbStairs(3))
```

---

```
... 5  
    3
```

---

14. A robot is located at the top-left corner of a  $m \times n$  grid. The robot can only move either down or right at any point in time. The robot is trying to reach the bottom-right corner of the grid. How many possible unique paths are there?

14.

---

```
def uniquePaths(m, n):  
    dp = [[1] * n for _ in range(m)]  
    for i in range(1, m):  
        for j in range(1, n):  
            dp[i][j] = dp[i-1][j] + dp[i][j-1]  
    return dp[m-1][n-1]  
print(uniquePaths(7, 3))  
print(uniquePaths(3, 2))
```

---

```
28  
3
```

---

15. In a string *S* of lowercase letters, these letters form consecutive groups of the same character. For example, a string like *s* = "abbxxxxzzy" has the groups "a", "bb", "xxxx", "z", and "yy". A group is identified by an interval [start, end], where start and end denote the start and end indices (inclusive) of the group. In the above example, "xxxx" has the interval [3,6]. A group is considered large if it has 3 or more characters. Return the intervals of every large group sorted in increasing order by start index.

15.

```
 def largeGroupPositions(s):  
    result = []  
    n = len(s)  
    i = 0  
    while i < n:  
        j = i  
        while j < n and s[j] == s[i]:  
            j += 1  
  
        if j - i >= 3:  
            result.append([i, j - 1])  
  
        i = j  
    return result  
print(largeGroupPositions("abbxxxxzzy"))  
print(largeGroupPositions("abc"))  
  
... [[3, 6]]  
    []
```

16. "The Game of Life, also known simply as Life, is a cellular automaton devised by the British mathematician John Horton Conway in 1970." The board is made up of an  $m \times n$  grid of cells, where each cell has an initial state: live (represented by a 1) or dead (represented by a 0). Each cell interacts with its eight neighbors (horizontal, vertical, diagonal) using the following four rules

Input: board = [[1,1],[1,0]]

Output: [[1,1],[1,1]]

16.

```
def gameOfLife(board):
    rows = len(board)
    cols = len(board[0])
    directions = [(-1,-1), (-1,0), (-1,1),
                  (0,-1),      (0,1),
                  (1,-1), (1,0), (1,1)]

    newBoard = [[0] * cols for _ in range(rows)]
    for i in range(rows):
        for j in range(cols):
            live = 0
            for dx, dy in directions:
                x = i + dx
                y = j + dy

                if 0 <= x < rows and 0 <= y < cols:
                    if board[x][y] == 1:
                        live += 1

            if board[i][j] == 1:
                if live == 2 or live == 3:
                    newBoard[i][j] = 1
            else:
                if live == 3:
                    newBoard[i][j] = 1

    return newBoard

board = [
    [0,1,0],
    [0,0,1],
    [1,1,1],
    [0,0,0]
]
print(gameOfLife(board))
board = [
    [1,1],
    [1,0]
]
print(gameOfLife(board))
```

```
[[0, 0, 0], [1, 0, 1], [0, 1, 1], [0, 1, 0]]
[[1, 1], [1, 1]]
```

17. We stack glasses in a pyramid, where the first row has 1 glass, the second row has 2 glasses, and so on until the 100<sup>th</sup> row. Each glass holds one cup of champagne. Then, some champagne is poured into the first glass at the top. When the topmost glass is full, any excess liquid poured will fall equally to the glass immediately to the left and right of it. When those glasses become full, any excess champagne will fall equally to the left and right of those glasses, and so on. (A glass at the bottom row has its excess champagne fall on the floor.) For example, after one cup of champagne is poured, the top most glass is full. After two cups of champagne are poured, the two glasses on the second row are half full. After three cups of champagne are poured, those two cups become full - there are 3 full glasses total now. After four cups of champagne are poured, the third row has the middle glass half full, and the two outside glasses are a quarter full, as pictured below.

17.

---

```
▶ def champagneTower(poured, query_row, query_glass):
    dp = [[0] * 101 for _ in range(101)]
    dp[0][0] = poured
    for row in range(query_row + 1):
        for glass in range(row + 1):
            if dp[row][glass] > 1:
                extra = (dp[row][glass] - 1) / 2
                dp[row + 1][glass] += extra
                dp[row + 1][glass + 1] += extra
                dp[row][glass] = 1
    return min(1, dp[query_row][query_glass])
print(champagneTower(1, 1, 1))
print(champagneTower(2, 1, 1))
```

---

```
... 0
     0.5
```